

Documento ABI de Llamadas al Sistema

Fase 2 — Modos de Ejecución y Syscalls | Sistemas Operativos

1. Resumen

Este documento define la interfaz binaria de aplicación (ABI) para las llamadas al sistema del kernel de la Fase 2. Las tareas en modo usuario solicitan servicios del kernel exclusivamente mediante esta interfaz, ejecutando la instrucción `svc #0` con los argumentos cargados en los registros ARM `r0`–`r3`.

El kernel valida que cada trap se originó desde el modo ARM USR (`CPSR[4:0] = 0x10`) antes de despachar. Los traps provenientes de cualquier modo privilegiado son rechazados con código de retorno `-1`.

2. ABI de Registros

Entrada (usuario → kernel):

Registro	Rol en la entrada	Notas
r0	ID de syscall	Sobreescrito con el valor de retorno al salir
r1	Argumento 1	Los argumentos no usados deben ser cero
r2	Argumento 2	
r3	Argumento 3	

Salida (kernel → usuario):

Registro	Rol en la salida
r0	Valor de retorno (<code>int32_t</code>). ≥ 0 en éxito; < 0 en error (ver Sección 5)
r1–r3	Preservados (el kernel no los modifica)

Instrucción de trap:

```
svc #0 @ trap canónico; el campo inmediato es ignorado por el despachador
```

3. Tabla de Identificadores de Syscall

Símbolo	ID Numérico	Propósito
<code>SYS_YIELD</code>	0	Reprogramación voluntaria del CPU; la tarea sigue siendo ejecutable
<code>SYS_EXIT</code>	1	Termina la tarea con un código de salida; no retorna
<code>SYS_WRITE</code>	2	Escribe <code>len</code> bytes desde el buffer de usuario al fd (UART/consola)
(inválido)	cualquier otro	Retorna <code>-1</code> de inmediato; sin cambios en el estado del kernel

4. Especificaciones de Syscalls

4.1 SYS_YIELD (id = 0)

Propósito: reprogramación cooperativa voluntaria del CPU.

r0 entrada	0 (SYS_YIELD)
r1–r3	Ignorados (establecer a 0)
r0 salida	0 en éxito

Comportamiento del kernel (en orden):

- Validar que el trap se originó desde modo USR.
- Marcar la tarea actual como READY (si no está TERMINATED).
- Ejecutar el planificador round-robin; seleccionar la siguiente tarea READY.
- Restaurar el contexto de la tarea seleccionada y retornar a USR mediante excepción de retorno.

4.2 SYS_EXIT (id = 1)

Propósito: terminar la tarea invocante. No retorna al caller.

r0 entrada	1 (SYS_EXIT)
r1	Código de salida (int32_t) — registrado en el PCB
r2–r3	Ignorados
r0 salida	Nunca retorna al caller

Comportamiento del kernel (en orden):

- Validar que el trap se originó desde modo USR.
- Guardar el pid saliente y el código de salida de r1 ANTES de llamar al planificador.
- Registrar en pcb_ext_table[pid]: exit_code, termination_reason = REASON_NORMAL_EXIT, last_syscall_rc.
- Marcar la tarea como TERMINATED; eliminarla del conjunto de tareas ejecutables.
- Emitir el trace de retorno con el pid saliente y rc = exit_code (antes de que current_pid cambie).
- Ejecutar el planificador. Si no quedan tareas, entrar en la política idle/halt documentada.
- Realizar la excepción de retorno a la nueva tarea (nunca regresa al caller).

4.3 SYS_WRITE (id = 2)

Propósito: escribir bytes desde un buffer en espacio de usuario a un descriptor de archivo (UART/console).

r0 entrada	2 (SYS_WRITE)
r1	fd — descriptor de archivo (válido: 1 = stdout/UART)
r2	buf — puntero al buffer en espacio de usuario
r3	len — número de bytes a escribir
r0 salida	Número de bytes escritos en éxito; código de error negativo en fallo

Comportamiento del kernel (en orden):

- Validar que el trap se originó desde modo USR.
- Validar `fd == 1`; retornar -2 si no.
- Validar que `buf+len` queda dentro de una región de usuario mapeada (P1: 0x00110000–0x00120000, P2: 0x00210000–0x00220000). Retornar -3 ante cualquier violación de puntero/rango, incluyendo desbordamiento aritmético.
- Escribir exactamente `len` bytes al UART byte a byte mediante `uart_putc()`, sin depender de null-terminator.
- Retornar `len` en éxito.

5. Valores de Retorno y Códigos de Error

Valor	Significado	Syscalls aplicables
<code>>= 0</code>	Éxito	<code>SYS_YIELD</code> retorna 0; <code>SYS_WRITE</code> retorna cantidad de bytes
<code>-1</code>	ID de syscall inválido	ID de syscall desconocido
<code>-2</code>	Descriptor o argumento inválido	<code>SYS_WRITE</code> : <code>fd != 1</code>
<code>-3</code>	Puntero de usuario inválido o violación de protección	<code>SYS_WRITE</code> : <code>buf</code> fuera de la región de usuario o desbordamiento

6. Envoltorio en Espacio de Usuario (`user/lib/syscall_api.h`)

Las tareas de usuario enlazan con los siguientes envoltorios. Se desaconseja el uso directo de ensamblador inline en el código de las tareas.

```
static int do_syscall(int id, int a1, int a2, int a3) {
    register int r0 __asm__ ("r0") = id;
    register int r1 __asm__ ("r1") = a1;
    register int r2 __asm__ ("r2") = a2;
    register int r3 __asm__ ("r3") = a3;
    __asm__ volatile ("svc #0" : "=r"(r0) : "r"(r0), "r"(r1), "r"(r2), "r"(r3) :
"memory");
    return r0;
}

void sys_yield(void)    { do_syscall(SYS_YIELD, 0, 0, 0); }
void sys_exit(int code) { do_syscall(SYS_EXIT, code, 0, 0); while(1){} }
int  sys_write(int fd, const char *buf, int len)
    { return do_syscall(SYS_WRITE, fd, (int)buf, len); }
```

7. Trazas MODE_SWITCH Requeridas

El kernel emite las siguientes líneas de traza al UART en cada cruce de syscall

```
@ Entrada:  MODE_SWITCH USER_TO_KERNEL pid=<n> reason=syscall id=<id>
@ Salida:   MODE_SWITCH KERNEL_TO_USER pid=<m> reason=syscall_return id=<id>
rc=<rc>
```

- pid en la entrada es la tarea invocante; pid en la salida puede ser diferente después de SYS_YIELD.
- Para SYS_EXIT: el trace de salida se emite con el pid de la tarea saliente y rc = exit_code, ANTES de que current_pid cambie al nuevo proceso.
- rc refleja el valor de r0 en el momento del retorno.
- Las líneas de traza se emiten incondicionalmente en ejecuciones de prueba y demostración.

8. Ruta de Ejecución a través del Vector SVC

El siguiente flujo describe la secuencia completa desde que el usuario ejecuta svc #0 hasta el retorno a USR:

- El usuario ejecuta svc #0 con el ID de syscall en r0 y los argumentos en r1–r3.
- La CPU entra en modo privilegiado en la entrada del vector SVC; el código de trap guarda/restaura el contexto de forma compatible con la semántica de la interrupción IRQ.
- El despachador lee r0, valida el rango del ID de syscall e invoca el handler correspondiente (yield, exit o write), o retorna -1 ante IDs desconocidos sin modificar el estado del kernel.
- El handler actualiza el estado del PCB/kernel según se especifica en la Sección 5.
- La ruta de retorno restaura la tarea de usuario apropiada (posiblemente distinta al caller tras SYS_YIELD) con r0 conteniendo el valor de retorno/error.

Comportamiento esperado: códigos de retorno determinísticos; sin corrupción del kernel por IDs de syscall inválidos, argumentos inválidos o punteros de usuario inválidos.

9. Checklist de Aceptación de Syscalls

✓	Criterio
☐	Al menos una tarea ejecuta write + yield repetidamente sin inestabilidad del kernel.
☐	exit elimina la tarea permanentemente del conjunto ejecutable (nunca es reanudada).
☐	Entradas inválidas a write retornan errores y no corrompen la memoria del kernel.
☐	Las trazas contienen pares de líneas syscall entrada/salida según la Sección 7 (filas 4 y 5 del catálogo MODE_SWITCH).
☐	Traps originados desde modo privilegiado son rechazados con código -1 sin modificar el estado del kernel.
☐	SYS_EXIT emite la traza de salida con el pid saliente antes de cambiar current_pid al nuevo proceso.
☐	Syscall ID desconocido retorna -1 sin modificar el estado del kernel.

10. Desviaciones y Limitaciones Conocidas

- **Regiones de usuario fijas:** la validación de puntero en SYS_WRITE usa rangos hardcodeados (P1: 0x00110000–0x00120000, P2: 0x00210000–0x00220000). En una fase futura estos rangos se derivarán dinámicamente del layout del PCB.
- **Un único fd soportado:** solo fd=1 (UART/stdout) es válido en la Fase 2. El soporte a múltiples descriptores queda diferido.
- **Sin límite de longitud explícito en SYS_WRITE:** se valida que buf+len no desborde la región mapeada, pero no se aplica un cap de longitud máxima adicional. Esto puede refinarse en fases futuras.